
ÍNDICES EN SQL SERVER

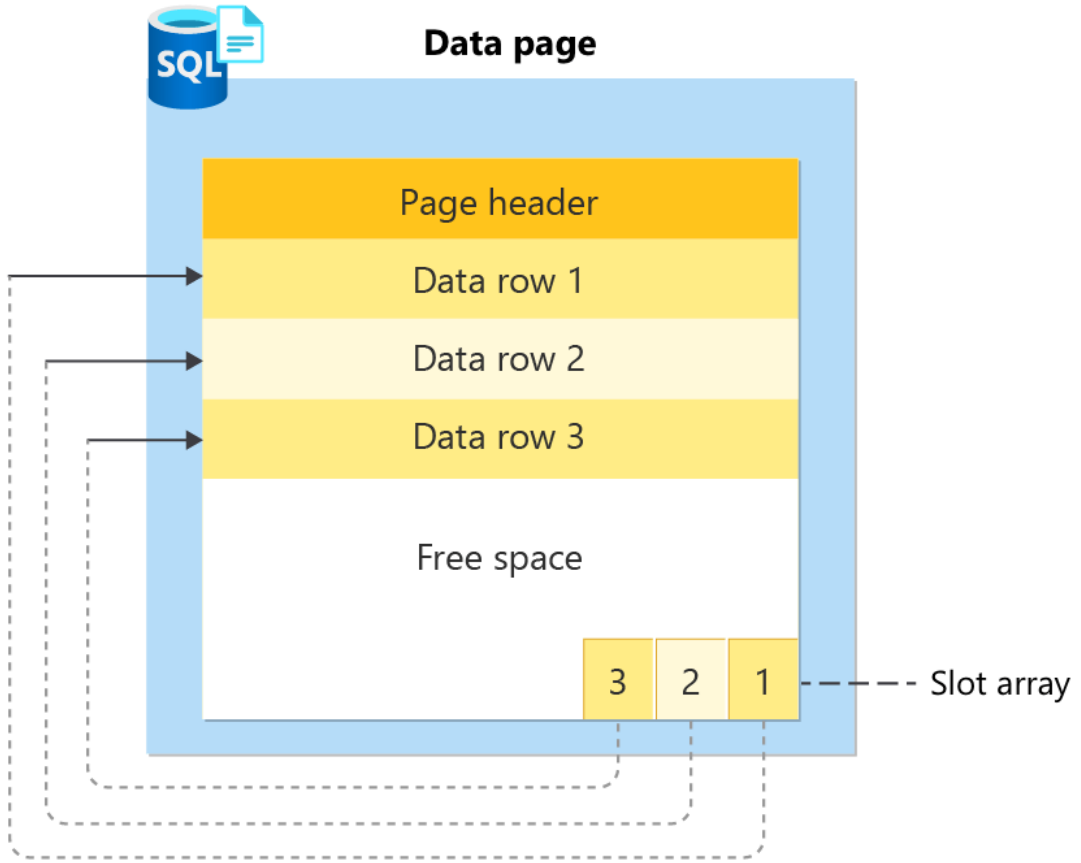
Aspectos clave:

- Gestión física de los datos: ficheros, páginas y extents
- Lecturas lógicas y físicas: el buffer pool
- Estructura B-tree: índices clustered y nonclustered
- La cláusula CREATE INDEX en detalle
- Índices COLUMNSTORE
- El planificador

Páginas y Extents

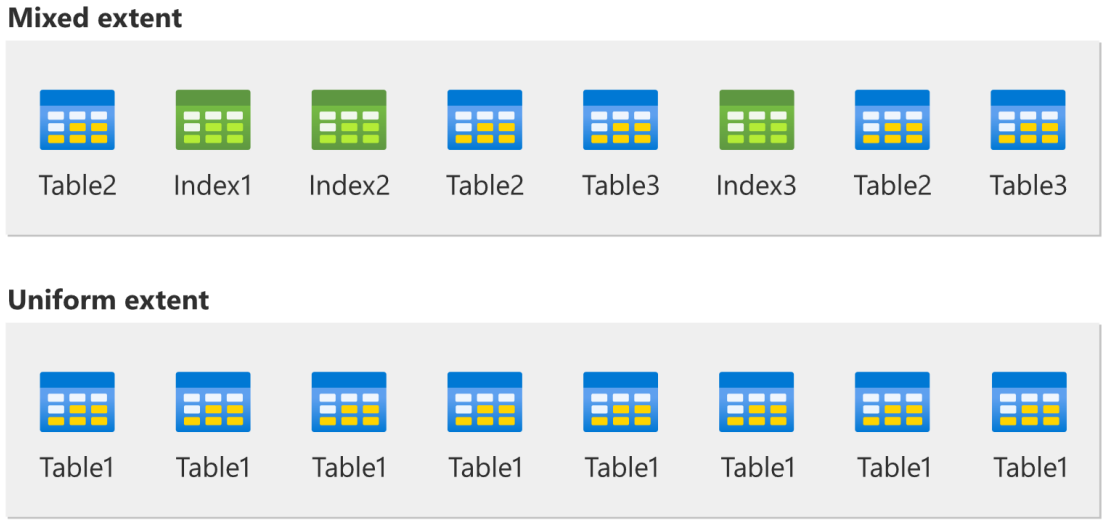
La página: unidad básica de almacenamiento

- Tamaño fijo: **8 KB = 8.192 bytes**.
- SQL Server **lee y escribe páginas completas**, nunca filas sueltas.
- Cada página se identifica por (id_fichero : nº_página), p. ej. 1:312.
- **128 páginas por MB**.



Extents

- Extent = 8 páginas contiguas = 64 KB.
- Es la unidad de asignación de espacio a los objetos.



Tipo	Descripción
Uniforme	Las 8 páginas pertenecen a un único objeto
Mixto	Hasta 8 objetos distintos comparten el extent

Lecturas lógicas y físicas

El buffer pool

- SQL Server **nunca** trabaja directamente sobre el disco: toda página se procesa **en memoria**, en el **buffer pool** (caché de páginas de datos).
- Las páginas se cargan en *buffers* de 8 KB.
- Toda lectura física acaba siendo también una lectura lógica. Por eso la métrica clave para comparar consultas es el número de lecturas lógicas: es estable e independiente del estado de la caché.

Concepto	Definición
Lectura lógica	Acceso a una página que ya está en el buffer pool
Lectura física	La página no está en memoria y hay que traerla del disco
Read-ahead	Lecturas físicas anticipadas: el motor predice qué páginas necesitará y las carga en bloques (hasta 64 páginas contiguas o más en Enterprise)
Página sucia	Página modificada en memoria y aún no escrita a disco

Escritura de páginas: checkpoint y lazy writer

Proceso	Función
Write-ahead logging (WAL)	El registro de log se escribe en disco antes que la página de datos modificada
Checkpoint	Escribe periódicamente páginas sucias a disco para acortar la recuperación
Lazy writer	Libera buffers cuando hay presión de memoria (escribe las sucias y expulsa las menos usadas)
Eager writer	Escribe páginas en operaciones minimally logged (bulk insert, SELECT INTO)

Scan frente a Seek

- Un *seek* sobre una tabla de 10 millones de filas suele costar **3-4 lecturas lógicas**; un *scan* puede costar **decenas de miles**.

Operador	Qué hace	Coste en lecturas
Table Scan	Recorre todo un heap	Todas las páginas
Clustered Index Scan	Recorre todo el nivel hoja del clustered	Todas las páginas de datos
Index Seek	Navega el B-tree hasta las claves buscadas	Profundidad del árbol + páginas hoja del rango
Key Lookup / RID Lookup	Por cada fila encontrada en un NC, va a buscar columnas al clustered/heap	~3-4 lecturas por fila

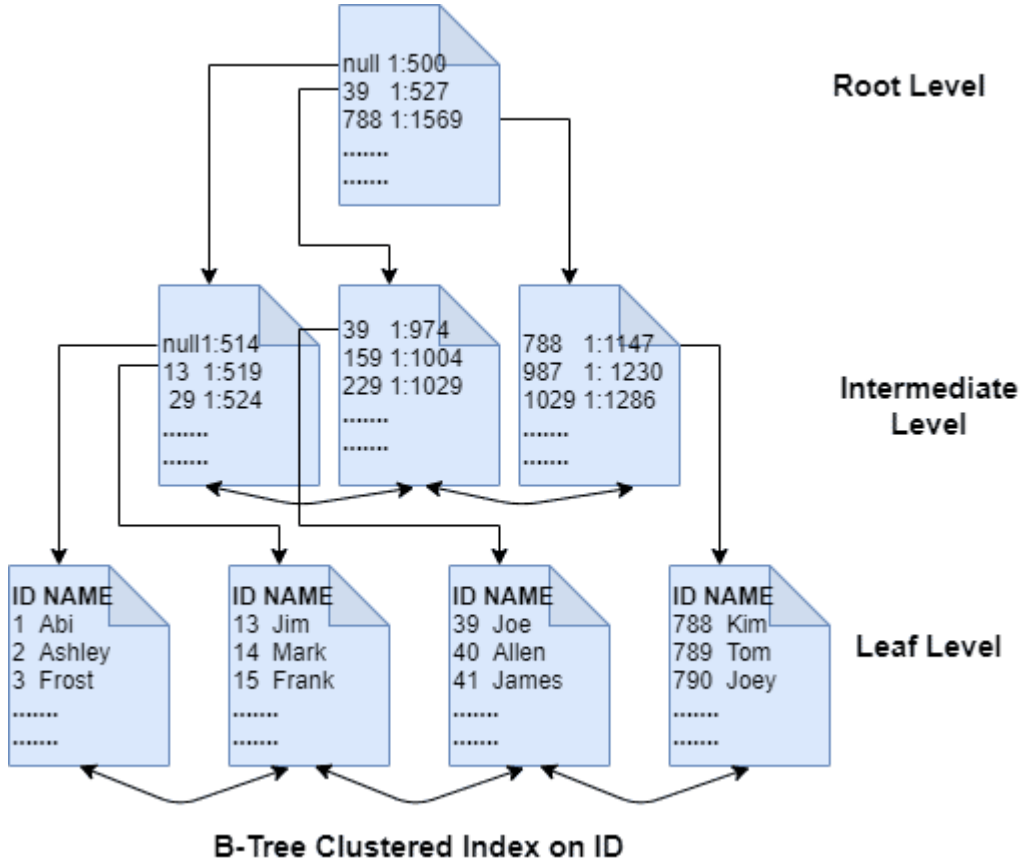
Índices clustered y nonclustered

Heap: tabla sin índice clustered

- Las filas **no tienen orden** y se localizan mediante el **RID** (*Row Identifier*) = fichero:página:slot (8 bytes).
- Se recorre por las páginas IAM.
- **Forwarded records:** si una fila crece y no cabe en su página, se mueve y deja un puntero → lecturas extra.
- **¿Cuándo tiene sentido un heap?** Tablas de staging de carga masiva que se leen completas y luego se truncan.

Estructura B-tree (árbol B+)

- Los índices *rowstore* se organizan como árboles equilibrados de páginas:
  - Todas las hojas están a la **misma profundidad**.
  - Las páginas de cada nivel están **doblemente enlazadas** (anterior/siguiente) → recorridos por rango eficientes.
  - Profundidad típica: 2-4 niveles incluso con millones de filas.



https://www.youtube.com/watch?v=K1a2Bk8NrYQ

Índice CLUSTERED

- **El nivel hoja SON los datos** de la tabla: la tabla se almacena ordenada por la clave.
- **Solo puede haber uno** por tabla.
- La PRIMARY KEY crea por defecto un clustered (si no existe otro).
- Si la clave no es única, SQL Server añade un **uniquifier** oculto de 4 bytes a los duplicados.

Propiedad	Motivo
Narrow (estrecha)	Se copia en todos los índices nonclustered
Unique (única)	Evita el uniquifier
Static (estática)	Cambiarla obliga a mover la fila y actualizar todos los NONCLUSTERED
Ever-increasing (creciente)	Inserciones al final → menos page splits y fragmentación

Índice NONCLUSTERED

- Estructura **separada** de los datos.
- El nivel hoja contiene **clave del índice + localizador de fila**:
  - Si la tabla tiene clustered → **la clave clustered**.
  - Si es heap → el **RID**.
- Hasta **999** índices nonclustered por tabla.
- **Key Lookup:** si la consulta pide columnas que no están en el NONCLUSTERED, por cada fila hay que navegar el CLUSTERED. Con muchas filas, el optimizador preferirá un *scan*.

La cláusula CREATE INDEX en detalle

UNIQUE, CLUSTERED / NONCLUSTERED y nombre

```
CREATE UNIQUE NONCLUSTERED INDEX UX_Clientes_NIF ON dbo.Clientes (NIF);
```

Elemento	Detalle
UNIQUE	No permite claves duplicadas. Varios NULL cuentan como duplicados (solo se admite uno), salvo con índice filtrado WHERE col IS NOT NULL Aporta información al optimizador (sabe que un seek devuelve como mucho una fila), además de garantizar integridad.
CLUSTERED	Ordena físicamente la tabla. Uno por tabla. Si la tabla ya tiene datos, la reconstruye entera
NONCLUSTERED	Valor por defecto si no se indica
nombre_indice	Único dentro de la tabla. Convención habitual: PK_, IX_, UX_, CIX_, CCI_

```
CREATE [ UNIQUE ] [ CLUSTERED | NONCLUSTERED ] INDEX nombre_indice
    ON <objeto> ( columna [ ASC | DESC ] [ ,...n ] )
    [ INCLUDE ( columna [ ,...n ] ) ]
    [ WHERE <predicado_filtro> ]
    [ WITH ( <opcion_indice> [ ,...n ] ) ]
    [ ON { esquema_particion ( columna ) | filegroup | "default" } ]
    [ FILESTREAM_ON { filegroup_filestream | esquema_particion | "NULL" } ]
[ ; ]

<opcion_indice> ::=
    PAD_INDEX = { ON | OFF }
  | FILLFACTOR = 1..100
  | SORT_IN_TEMPDB = { ON | OFF }
  | IGNORE_DUP_KEY = { ON | OFF }
  | STATISTICS_NORECOMPUTE = { ON | OFF }
  | STATISTICS_INCREMENTAL = { ON | OFF }
  | DROP_EXISTING = { ON | OFF }
  | ONLINE = { ON [ ( WAIT_AT_LOW_PRIORITY (...) ) ] | OFF }
  | RESUMABLE = { ON | OFF }
  | MAX_DURATION = <minutos> [ MINUTES ]
  | ALLOW_ROW_LOCKS = { ON | OFF }
  | ALLOW_PAGE_LOCKS = { ON | OFF }
  | OPTIMIZE_FOR_SEQUENTIAL_KEY = { ON | OFF }
  | MAXDOP = n
  | DATA_COMPRESSION = { NONE | ROW | PAGE } [ ON PARTITIONS ( ... ) ]
  | XML_COMPRESSION = { ON | OFF } [ ON PARTITIONS ( ... ) ]
```

ON objeto (columnas **ASC | DESC**)

```
CREATE INDEX IX_Pedidos_Cliente_Fecha ON dbo.Pedidos (ClienteID, Fecha DESC);
```

- El objeto puede ser una **tabla** o una **vista** (vista indexada: el primer índice debe ser UNIQUE CLUSTERED y la vista WITH SCHEMABINDING).
- **Orden de las columnas = lo más importante del diseño**: el índice solo permite *seek* sobre el **prefijo izquierdo** de la clave.
- **ASC / DESC**: relevante solo con varias columnas y ORDER BY mixto (p. ej. ORDER BY ClienteID ASC, Fecha DESC). Con una sola columna, el motor puede recorrer el índice hacia atrás.
- Regla práctica: **igualdad primero, desigualdad (rangos) después**.
- Límites: 32 columnas clave; 900 bytes (clustered) / 1.700 bytes (NC) desde SQL Server 2016.
- No admiten columna clave: VARCHAR(max), XML, TEXT, IMAGE.

INCLUDE

```
CREATE INDEX IX_Pedidos_ClienteID
ON dbo.Pedidos (ClienteID)
INCLUDE (Fecha, Importe, Observaciones);
```

- Solo para **NONCLUSTERED**.
- Las columnas se guardan **solo en la hoja**: el árbol superior se mantiene compacto.
- No cuentan para el límite de 32 columnas ni de 1.700 bytes.
- Permite tipos NVARCHAR(max), VARBINARY(max), XML (no TEXT/IMAGE).
- Una columna no puede estar a la vez en la clave y en INCLUDE.
- *No incluir en INCLUDE toda la tabla:*
  - *cada columna incluida duplica datos y encarece las escrituras.*

WHERE: índices filtrados

```sql
-- Solo pedidos pendientes (5 % de la tabla)
CREATE INDEX IX_Pedidos_Pendientes
ON dbo.Pedidos (FechaEntrega)
INCLUDE (ClienteID)
WHERE Estado = 'P';

-- Unicidad ignorando NULL
CREATE UNIQUE INDEX UX_Clientes_Email
ON dbo.Clientes (Email)
WHERE Email IS NOT NULL;
```

- **Ventajas:**
  - menor tamaño, menor mantenimiento, estadísticas más precisas.
- **Restricciones:**
  - Solo NONCLUSTERED sobre tablas (no vistas).
  - Predicados simples: =, <>, >, <, IN, IS NULL, IS NOT NULL combinados con AND. No admite OR, LIKE, funciones ni columnas calculadas.
  - Las consultas **parametrizadas** pueden no usarlo (el optimizador no sabe si el valor del parámetro cae dentro del filtro) → OPTION (RECOMPILE) o literales.
  - Requiere SET ANSI_NULLS, QUOTED_IDENTIFIER, etc. en ON en las sesiones que modifican la tabla.

Sargability

- **SARG** = *Search Argument*. Un predicado es sargable si SQL Server puede usarlo para posicionarse en el árbol B+.
- La regla es una sola: **la columna indexada debe aparecer sola a un lado del operador de comparación.** En cuanto la envuelves en una función o le haces aritmética, el motor ya no puede saber a qué rama del árbol ir y se ve obligado a evaluar fila por fila.

Catálogo de antipatrones

Antipatrón	Escritura sargable
`WHERE YEAR(Fecha) = 2023`	`WHERE Fecha >= '2023-01-01' AND Fecha < '2024-01-01'`
`WHERE CONVERT(VARCHAR, Fecha, 112) = '20230615'`	`WHERE Fecha >= '2023-06-15' AND Fecha < '2023-06-16'`
`WHERE DATEDIFF(DAY, Fecha, GETDATE()) <= 30`	`WHERE Fecha >= DATEADD(DAY, -30, CAST(GETDATE() AS DATE))`
`WHERE ISNULL(Estado,'') = 'Pendiente'`	`WHERE Estado = 'Pendiente'` (y arregla la nulabilidad)
`WHERE Importe * 1.21 > 1000`	`WHERE Importe > 1000 / 1.21`
`WHERE UPPER(Nombre) = 'GARCIA'`	`WHERE Nombre = 'Garcia'` (con collation case-insensitive)
`WHERE Codigo LIKE '%745123'`	Sin solución con B+ tree. Valorar Full-Text o columna calculada invertida.
`WHERE Cliente = 1 OR Producto = 88`	`UNION` de dos consultas
`WHERE Col IN (SELECT ...)` con función	Reescribir con `JOIN` o `EXISTS`

Checklist de diseño de un índice

- ¿Qué consulta concreta quiero acelerar? (Si no puedes nombrarla, no crees el índice.)
- ¿Qué columnas filtra con igualdad? → clave, primero.
- ¿Qué columna filtra por rango? → clave, después.
- ¿Qué columnas ordena o agrupa? → clave, en ese orden.
- ¿Qué columnas devuelve? → INCLUDE.
- ¿Existe ya un índice parecido que pueda ampliar en lugar de duplicar?
- ¿Cuántas escrituras por segundo recibe esta tabla?
- ¿He medido el antes? ¿He medido el después?

- Índices COLUMNSTORE

• Rowstore frente a COLUMNSTORE

	Rowstore	Columnstore
Unidad de almacenamiento	Fila en páginas de 8 KB	Segmentos de columna
Lee	Todas las columnas de la fila	Solo las columnas de la consulta
Compresión	Opcional (ROW/PAGE)	Siempre, típicamente hasta 10x
Ideal para	OLTP: búsquedas puntuales, actualizaciones	Analítica: agregaciones sobre millones de filas
Modo de ejecución típico	Fila a fila (row mode)	Batch mode (~900 filas por lote, vectorizado)

¿Por qué es tan rápido?

- **Lectura por columnas:** SELECT SUM(Importe) lee solo el segmento Importe.
- **Compresión:** diccionarios, codificación por valor, *run-length encoding* → menos páginas, más datos en memoria.
- **Segment elimination:** con WHERE Fecha >= '2026-01-01' se descartan los segmentos cuyo máximo es anterior, sin leerlos.
- **Batch mode:** procesamiento vectorizado por lotes, aprovecha las CPU modernas.
- **Aggregate pushdown:** agregados calculados directamente durante el scan.

•Clustered y nonclustered COLUMNSTORE

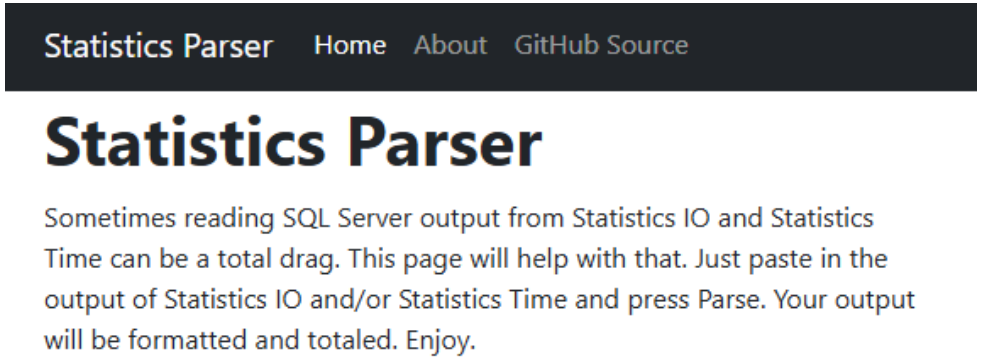
	Clustered columnstore (CCI)	Nonclustered columnstore (NCCI)
Qué es	La tabla entera en formato columnar	Copia columnar de algunas columnas sobre una tabla rowstore
Nº por tabla	1 (sustituye al clustered rowstore)	1
Escenario	Tablas de hechos de un DWH	Analítica operacional (HTAP) sobre tablas OLTP
Índices B-tree adicionales	Sí, desde SQL Server 2016 (para búsquedas y unicidad)	— (la tabla ya tiene sus índices rowstore)
Filtrado	No	Sí (WHERE), p. ej. solo datos "fríos"

Medición de rendimiento

Medir lecturas: SET STATISTICS

```
SET STATISTICS IO ON;
SET STATISTICS TIME ON;
```

https://statisticsparser.com/?lang=en



Campo	Interpretación
Scan count	Nº de recorridos (seeks o scans) iniciados sobre el objeto
logical reads	Páginas leídas de memoria → la métrica a optimizar
physical reads	Páginas leídas de disco de forma síncrona
read-ahead reads	Páginas cargadas por anticipación
lob logical reads	Páginas LOB leídas

Medición del Execution Plan

- El plan gráfico se lee de derecha a izquierda y de arriba abajo: los operadores de la derecha son los que primero acceden a los datos, y el flujo va confluyendo hacia el resultado final a la izquierda.
- El grosor de las flechas que conectan operadores es proporcional al número de filas que circulan por ellas; una flecha muy gruesa donde esperabas pocas filas es una pista visual inmediata de una mala estimación.
- Cada operador muestra un porcentaje: es su coste estimado relativo respecto al coste total del lote. El operador con el porcentaje más alto es el candidato natural para optimizar, aunque conviene recordar que ese número es una estimación del optimizador, no un tiempo real medido.
- Dos gestos en SSMS valen más que memorizar iconos:
  - pasar el ratón por encima de un operador abre un tooltip con filas estimadas frente a reales, coste, modo de ejecución (row o batch) y predicados
  - tecla F4 (o botón derecho, Properties) abre el panel completo de propiedades del operador seleccionado

Pistas visuales y advertencias

- Más allá del icono, el plan comunica información con marcas visuales que conviene reconocer:
  - El signo de exclamación amarillo o rojo sobre un operador indica una advertencia: conversión implícita, concesión de memoria excesiva o insuficiente (spill a tempdb), o estadísticas ausentes. Siempre merece pasar el ratón por encima para leerla.
  - La doble flecha amarilla en la esquina del icono indica que el operador se ejecutó en paralelo entre varios núcleos.
  - Una gran discrepancia entre Estimated Number of Rows y Actual Number of Rows (visible en el tooltip o con F4) es la firma clásica de estadísticas desactualizadas o de parameter sniffing: el optimizador planificó para un número de filas y se encontró con otro muy distinto.
  - Las flechas gruesas señalan por dónde circula el grueso de las filas. Una flecha gruesa que alimenta un Key Lookup o un Sort es una señal de alarma habitual.

Operadores de acceso a datos

Operador	Qué significa / cuándo aparece	Señal
Clustered Index Seek	Navega directamente al punto del índice clustered que cumple el predicado. Es el acceso más eficiente cuando se buscan pocas filas por una columna indexada.	**Buena**
Index Seek (NonClustered)	Igual que el anterior pero sobre un índice nonclustered. Si el índice no cubre todas las columnas pedidas, suele ir acompañado de un Key Lookup.	**Buena**
Clustered Index Scan	Recorre toda la tabla (el clustered ES la tabla). Correcto si se necesita la mayoría de las filas; sospechoso si esperabas recuperar solo unas pocas.	**Neutra**
Index Scan (NonClustered)	Recorre entero un índice nonclustered. Más barato que el scan del clustered si el índice es estrecho, pero sigue siendo un recorrido completo.	**Neutra**
Table Scan	Recorre un heap completo (tabla sin índice clustered). Casi siempre indica que falta un índice o que la tabla debería tener clave clustered.	**Investigar**
Columnstore Index Scan	Lectura sobre un índice columnstore. En agregaciones sobre grandes volúmenes es lo deseable; habilita compresión y modo batch.	**Buena**
RID Lookup	Salto a un heap para recuperar columnas que el índice nonclustered no tiene. Equivalente al Key Lookup pero sobre heap.	**Investigar**
Key Lookup	Salto al clustered, una fila cada vez, para traer columnas que faltan en el índice nonclustered. Barato para pocas filas, catastrófico repetido miles de veces. Se elimina con un índice covering (INCLUDE).	**Investigar**

Lectura rápida

	Comprobación
1	¿Hay Table Scan o Clustered Index Scan donde esperabas un Seek? → posible índice ausente o predicado non-sargable.
2	¿Aparece Key Lookup o RID Lookup con flecha gruesa? → candidato a índice covering con INCLUDE.
3	¿Estimated Rows vs Actual Rows difieren mucho? → estadísticas o parameter sniffing.
4	¿Hay un Sort costoso? → ¿puede un índice ordenado evitarlo?
5	¿El operador de mayor % de coste es el que esperabas? → ahí se concentra el esfuerzo de optimización.
6	¿Hay triángulos de advertencia (amarillo/rojo)? → léelos: conversión implícita, spill a tempdb, falta de estadísticas.

Tres planes de ejecución

- **Estimated Execution Plan - Ctrl + L**
  - **No ejecuta la consulta.** Solo compila y muestra lo que el optimizador *piensa* hacer.
  - Útil cuando la consulta tarda 40 minutos o hace un DELETE que no quieres lanzar.
  - Solo contiene números estimados. No hay "filas reales" con las que comparar.
  - Limitación: falla si la consulta referencia objetos temporales creados en el mismo lote.
- **Actual Execution Plan - Ctrl + M**
  - Es un **modo persistente**: se queda activado hasta que lo desactivas.
  - Ejecuta la consulta y añade la pestaña **Execution plan** junto a Results y Messages.
  - Contiene estimaciones **y** valores reales. La comparación entre ambos es la mitad del diagnóstico.
  - Añade una pequeña sobrecarga de captura; suficientemente pequeña para no invalidar la medida.
- **Live Query Statistics**
  - Muestra el plan animándose durante la ejecución: contadores de filas subiendo en tiempo real.
  - Excelente **valor didáctico**: se ve físicamente dónde se atasca la consulta.
  - Es el único modo que sirve para diagnosticar una consulta que *aún* está corriendo.
  - Coste de observación mayor. No lo uses para cronometrar, úsalo para entender.

Planificador

Optimizador de consultas

- El optimizador de consultas de SQL Server (el **Query Optimizer**, a menudo llamado QO) es un optimizador **basado en costes**: su trabajo es tomar tu consulta declarativa (dices *qué* quieres, no *cómo*) y decidir el plan de ejecución físico más barato para obtenerlo.
- Cuando envías una consulta, pasa por varias fases antes de ejecutarse:
  - **parsing** (análisis sintáctico, se construye un árbol),
  - **binding/algebrización** (se resuelven nombres de tablas y columnas, tipos, se valida que todo existe),
  - **optimización** (el QO genera y evalúa planes candidatos y elige uno) y
  - **ejecución**. El optimizador actúa en la tercera fase, y su salida es el *execution plan*.
- Un punto conceptual importante: SQL Server **no busca el plan óptimo absoluto**. Buscar el mejor plan matemáticamente posible sería más caro que ejecutar la consulta. El QO busca un plan **suficientemente bueno en un tiempo razonable**; se detiene en cuanto encuentra uno con coste aceptable o alcanza su límite de esfuerzo. Es una optimización pragmática, no exhaustiva.

Estadísticas

- Esta es probablemente la pieza más importante de entender, porque es la que más problemas causa en la práctica.
- El QO no mira los datos reales al optimizar; mira **estadísticas** sobre esos datos. Una estadística es principalmente un **histograma** que describe cómo se distribuyen los valores de una columna: cuántas filas hay en cada rango de valores, cuántos valores distintos, etc.
- Con esas estadísticas el optimizador hace **estimaciones de cardinalidad**: predice cuántas filas devolverá cada operación. "Este WHERE Fecha = '2025-06-01' devolverá ~340 filas; este join producirá ~12.000." Esas estimaciones son la base de todos los cálculos de coste posteriores.
- De aquí sale la causa número uno de planes malos: **estadísticas desactualizadas o sesgadas**. Si las estadísticas dicen "340 filas" pero en realidad son 3.400.000, el optimizador elegirá una estrategia diseñada para pocos datos (por ejemplo, un Nested Loops con Key Lookups) que se vuelve desastrosa con muchos. Por eso mantener las estadísticas al día (UPDATE STATISTICS, auto-update activado, sp_updatestats) es tan crítico como los propios índices.

Costes

- El modelo de costes
  - Cada operador físico (scan, seek, join, sort, hash…) tiene un **coste estimado** que combina un componente de **E/S** (accesos a disco) y uno de **CPU**. El coste es un número abstracto, sin unidades reales de tiempo; es solo una moneda interna para comparar planes entre sí. Cuando en el plan ves "coste 78%", es la porción de ese coste estimado que se lleva un operador respecto al total del lote.
  - El QO enumera planes alternativos (¿qué join usar? ¿en qué orden unir las tablas? ¿qué índice? ¿scan o seek?) y elige el de menor coste total estimado. El **orden de los joins** es una de las decisiones más importantes y de las más costosas de explorar, porque el número de combinaciones crece exponencialmente con el número de tablas.
- El optimizador elige, entre otras cosas:
  - **Método de acceso**: *Table/Index Scan* (leer todo) vs *Index Seek* (ir directo). No siempre el seek es mejor: si vas a leer gran parte de la tabla, un scan es más eficiente. El QO estima la selectividad para decidir, y ahí entra el **tipping point**, el umbral a partir del cual prefiere scan aunque haya un índice disponible.
  - **Algoritmo de join**: *Nested Loops* (ideal para pocas filas en la entrada externa), *Merge Join* (ideal si ambas entradas ya vienen ordenadas por la clave de join), *Hash Join* (ideal para grandes volúmenes sin orden previo). Cuál elige depende del tamaño estimado y de si los datos ya están ordenados.
  - **Estrategia de agregación y ordenación**: *Stream Aggregate* vs *Hash Aggregate*, si necesita un *Sort* explícito, etc.
- SQL Server cachea el plan resultante en el plan cache y lo reutiliza para ejecuciones posteriores de la misma consulta. Esto ahorra mucho, pero abre uno de los problemas más famosos: el parameter sniffing.
- Para consultas muy simples (un SELECT por clave primaria), el QO ni se molesta en optimizar en profundidad: aplica un trivial plan directamente. Para las demás pasa por fases de optimización